

Java Annotations

meta-dati nei sorgenti Java

Le **annotazioni** in Java sono un sistema con cui il programmatore fornisce informazioni aggiuntive (**metadati**) su un elemento che sta dichiarando nel codice

```
@AnnotazioneBase
```

```
@AnnotazioneStrutturata("someinfo")
```

```
@AnnotazioneStrutturata(elem1="val1", elem2="val2")
```

Java Annotations – I

Il linguaggio Java di suo definisce alcune annotazioni predefinite, ma nuove annotazioni possono essere definite (come altri costrutti del linguaggio) dagli sviluppatori. Nel nostro caso avremo a che fare con annotazioni definite nei package inclusi nel progetto tramite il **pom.xml**.

Le annotazioni sintatticamente sono contraddistinte dal simbolo iniziale **@** e caratterizzate da un **nome** convenzionalmente scritto in **UpperCamelCase**. Alcune annotazioni richiedono una ulteriore specificazione di **coppie chiave-valore** che vengono in quel caso indicate fra parentesi tonde.

Quando un'annotazione prevede **un solo valore**, convenzionalmente associato alla chiave **value**, è possibile indicare tale valore fra parentesi tonde senza indicare la chiave.

È possibile annotare diversi costrutti del linguaggio: una classe, un campo, un metodo, un parametro di un metodo...

```
@AnnotazioneClasse class MyClass {  
    @AnnotazioneVar private int myVar;  
    @AnnotazioneMetodo  
    public void myMethod(@AnnotazioneParam String arg) {  
        ...  
    }  
}
```

Java Annotations – II

Le annotazioni sono **puramente dichiarative**: non **fanno** qualcosa, **forniscono** informazioni sull'elemento annotato.

Tali informazioni possono essere utili a diversi «attori» nel processo di sviluppo ed esecuzione:

- ad altri sviluppatori o sviluppatrici che devono metter mano al codice o comunque leggerlo, a mo' di segnalazione (es.: `@Deprecated`)
- al compilatore, per effettuare verifiche (es.: `@Override`; se il metodo così annotato non è effettivamente un override di un omonimo nella classe base, il compilatore segnala un errore)
- a servizi runtime, per attuare determinati comportamenti sugli elementi annotati.

Quest'ultimo punto è quello che rende le annotazioni estremamente rilevanti già in Jakarta Web Profile, ma ancor più in Spring e Spring Boot.

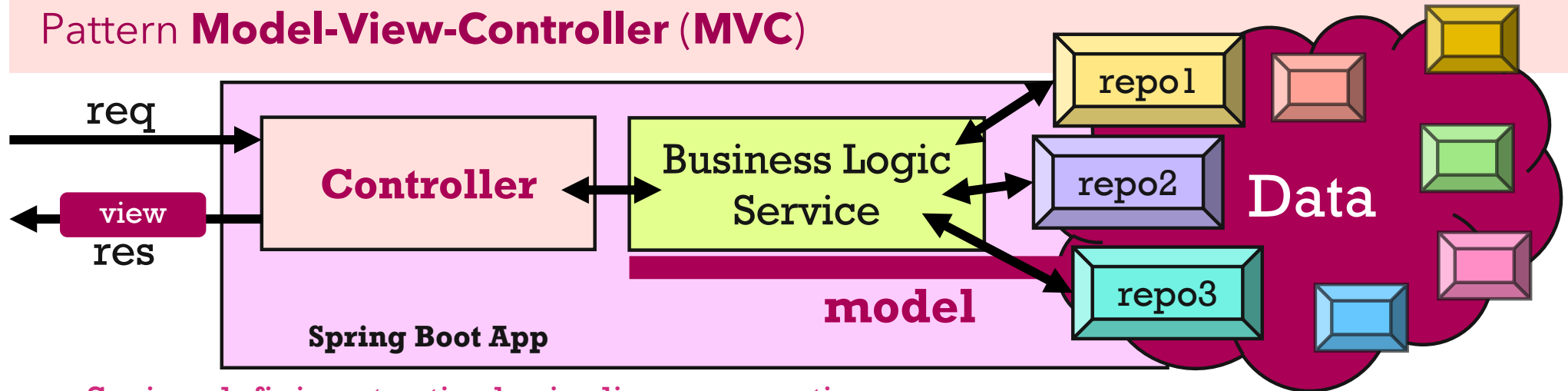
Il framework Spring infatti è appunto quel servizio runtime che, grazie alla **Java Reflection API**, legge le annotazioni apposte sulle nostre classi, metodi, parametri, campi, ecc... e attraverso di esse capisce come, dove e quando utilizzarli, come invocarne le funzionalità, e come dotarli di comportamenti aggiuntivi forniti da Spring stesso. Potremo entrare in dettagli più specifici quando analizzeremo le singole annotazioni pertinenti ai nostri obiettivi.

Possiamo però dire che l'uso delle annotazioni in Spring si avvicina a tutti gli effetti ad una **estensione del linguaggio Java**.

Spring-Boot

Caratteristiche principali

Pattern **Model-View-Controller (MVC)**



Spring definisce tre tipologie di componenti

@Controller @Service @Repository

Dependency Injection e Inversion of Control

Spring applica la **inversion of control**.

- Supponiamo che al componente X serva il componente Y
- **Controllo diretto**: X crea un'istanza di Y
- **Controllo rovesciato**: il framework crea un'istanza di Y e la «inietta» a tutti coloro che la richiedono, fra cui X
- **Dependency injection**: X «chiede» un oggetto Y nel suo costruttore, il framework glielo passa

Il pattern MVC in Spring – I

Spring struttura le applicazioni con il pattern Model-View-Controller. In questo pattern l'applicazione è costituita da **più controller**, ciascuno dei quali è responsabile per rendere «fruibile» una parte del **modello** (che contiene i dati e la loro logica) tramite una o più **viste**.

Il punto di ingresso dell'applicazione è sempre uno dei **controller**: è il controller che riceve la richiesta HTTP e la elabora. In Spring i componenti che svolgono il ruolo di controller sono **classi** identificate dall'**annotazione @Controller**

- Il controller fa da «mediatore» fra una o più **view** (ossia la presentazione delle informazioni, che include sia il modo in cui sono rappresentate all'esterno, sia le possibili azioni che è possibile operare dall'esterno per chiedere modifiche) e gli aspetti **model** (ossia la business logic e i repository dati, che detengono le informazioni, la loro strutturazione, i loro vincoli di coerenza, e sanno come effettuare eventuali aggiornamenti ai dati stessi) che sono di sua pertinenza
- Il controller è dunque responsabile dell'**API**, che possiamo vedere come la **descrizione della view** che il controller mette a disposizione

La **view** in un'applicazione di questo tipo è la «vista» su un certo sottoinsieme coerente di dati.

- Se tradizionalmente il termine **vista** può essere inteso in senso letterale, ossia una *presentazione grafica*, in questo contesto l'accezione è più ampia: una vista è **un punto di vista da cui guardare ai dati** (quindi un insieme di oggetti JSON è a tutti gli effetti una vista) e di questo fanno parte anche i modi messi a disposizione per **richiedere aggiornamenti** dei dati.
- Come già detto dunque, ciascun **controller** realizza una **vista**, e la descrive tramite la sua **API**: quali oggetti JSON restituisce in base a quali richieste, quali aggiornamenti ai dati mette a disposizione, quali informazioni bisogna fornire per richiederli.

Il pattern MVC in Spring – II

Il **model** dell'applicazione contiene la business logic e l'accesso ai dati, occupandosi anche della loro persistenza. Esso conosce i dati e la loro strutturazione, nonché i loro vincoli di coerenza, è dunque in grado di organizzarli e di aggiornarli quando necessario. In Spring i dati in quanto tali sono organizzati in componenti `@Repository`, strettamente collegati al tema della **persistenza**.

- I dati possono essere acceduti dai **controller** così come sono (quindi con un'organizzazione DB-like) oppure tramite la mediazione di **servizi di Business Logic** che aggregano e strutturano i dati, ne garantiscono la consistenza, li «gestiscono» secondo una «logica d'impresa» specifica, e possono offrire diverse prospettive su di essi.
- Per questa ragione nel **model** dell'MVC di Spring troviamo anche i componenti `@Service`, che permettono al controller di non occuparsi (ad esempio) dei dettagli di come i dati sono rappresentati, e forniscono operazioni di più alto livello.

Inversion of Control

L'**inversion of control** (rovesciamento del controllo) è un pattern architetturale attualmente utilizzato in moltissimi framework. Per comprenderlo è utile prima capire cosa si intende per **controllo diretto**.

Immaginiamo due componenti software X e Y (classi, moduli, o anche altro). X ha bisogno dei servizi di Y.

In caso di controllo diretto, è responsabilità dello sviluppatore **creare un'istanza di Y** (anche, volendo, nel codice di X stesso) e passarla a **X** perché possa utilizzarla.

Nel **controllo rovesciato** esiste un framework che mantiene tutti i «servizi»^(*) (come Y) le cui funzionalità possono essere richieste da altri, e li fornisce a chi dichiara di necessitarne.

Una caratteristica importante è che il framework potrebbe anche esporre versioni **astratte** dei servizi, il che gli permette di fornirne implementazioni diverse a seconda del contesto, nonché di modificarne l'implementazione senza «scossoni» per le classi che li utilizzano, o di fornirne versioni «mock» quando si sta effettuando il testing.

Il **controllo rovesciato** spesso viene realizzato tramite un meccanismo detto **dependency injection**. Il **componente** che vuole usare determinati servizi lo dichiara, ad esempio specificandoli come **parametri formali** del proprio costruttore. Poiché è il framework a costruire il **componente**, nel momento in cui lo fa il framework gli passa le **istanze** dei servizi che ritiene appropriate e che ha già creato in precedenza. Questo meccanismo è appunto la «iniezione delle dipendenze». Esso permette un minore accoppiamento fra l'utilizzatore e il fornitore di un servizio.

(*) in questo contesto il termine «servizio» è usato in senso astratto e non va identificato con i @Service di Spring Boot, che possono beneficiare della **dependency injection** in entrambi i ruoli

Il @RestController di Spring Boot

Rispondere a una richiesta HTTP di base

il **controller** è una classe annotata con @RestController

un **request handler** è un **metodo** di un **controller** annotato con @RequestMapping o con uno degli shortcut @GetMapping, @PostMapping...

@RequestMapping e i relativi shortcut ammettono **proprietà** che permettono di filtrare le richieste

- **value**: un pathname (String) o un insieme di pathname (array di String).
- **method**: il method della richiesta (una fra le costanti RequestMethod.GET, RequestMethod.POST,...)
- **params**: uno o più query params (String) che la request dovrà avere; è possibile anche vincolarne il valore con una stringa della forma "param=value"
- **headers**: uno o più header (String) che la request dovrà avere, anche qui si può vincolare il valore con una stringa della forma "header=value"

@GetMapping equivale a @RequestMapping(method=RequestMethod.GET)

- gli **shortcut** non ammettono la proprietà **method** perché è già implicita

È possibile applicare @RequestMapping a tutta la classe **controller**; il pathname verrà richiesto come prefisso dei pathname dei singoli **request handler**.

ResponseEntity<T>

Inviare risposte di tipo **application/json**

La classe `ResponseEntity` permette ai **request handler** di restituire in maniera relativamente semplice oggetti attorno a cui Spring **confezionerà** una risposta HTTP che, **di default**, sarà di tipo **application/json**

La classe `ResponseEntity` prevede dei **metodi statici** di convenienza per "costruire" la risposta. Vediamo alcuni esempi più comunemente usati:

- **`ResponseEntity.ok(data)`** costruisce una risposta con status code 200 e come **body** l'oggetto **data**.
- **`ResponseEntity.notFound().build()`** costruisce una risposta con status code 404. Perché serve l'ulteriore chiamata a **`build()`**? Perché **`notFound`** non restituisce una richiesta compiuta ma una richiesta parziale, sulla quale si potrebbero chiamare ulteriori configurazioni, a differenza di **`ok()`** che restituisce una richiesta compiuta.
- **`ResponseEntity.status(200).headers(httpHeaders).body(data)`** costruisce una risposta con status code 200 e come **body** l'oggetto **data**, al contempo permettendo di configurare gli **header** tramite un oggetto `HttpHeaders`, che è un insieme di coppie proprietà/valore

Attenzione: Spring determina **quali campi** di un oggetto siano da **serializzare e deserializzare** a partire dai **getter pubblici** della classe (mentre è indipendente dalla presenza dei corrispettivi **setter**).

parametri, header, body della request

ricevere le proprietà della richiesta nel request handler

Un **response handler** può specificare come parametri del metodo quelle proprietà che desidera conoscere dell'oggetto **request**.

Tramite opportune **annotazioni** comunicherà a Spring quale proprietà della request corrisponde a ciascun parametro.

- **@RequestParam** indica un parametro della query string. Se il nome del parametro della query string coincide con il nome del parametro formale del **request handler**, non serve specificare ulteriormente.
- **@RequestParam(name="parName")** fa corrispondere il parametro formale del **request handler** al parametro parName nella query string.
- **@RequestParam(required=false)** permette di esprimere il fatto che il parametro della query string può essere assente.
- **@PathVariable** indica un segmento parametrico del path, che deve essere indicato nel **pathname** del **@RequestMapping** fra parentesi **graffe** ({ }) con un **nome**. Tale nome dovrà coincidere con quello del parametro formale del **request handler**.
- **@RequestBody** indica il **body** della richiesta; il parametro formale del **request handler** deve essere del **tipo corretto** per permettere la **deserializzazione** della JSON string nella richiesta.
- **@RequestHeader(HttpHeaders.HEADER)** fa sì che il parametro formale contenga il **valore** dell'header specificato; HttpHeaders definisce costanti statiche corrispondenti ai **nomi** degli header HTTP.

@Service in Spring Boot

Gestire i dati e la business logic

Un **servizio di business logic** è una classe annotata con @Service.

Spring crea **una istanza** (e solo una) per ciascun servizio, dunque è importante che la classe posseda un costruttore **senza parametri**(*)

La classe in questione può essere fatta in qualsiasi modo si ritiene opportuno. I principi che ne guidano la progettazione e sviluppo hanno più a che vedere con i principi di una buona programmazione object-oriented che con Spring! L'idea è che un **servizio** offra alle altre componenti dell'applicazione un insieme di **funzionalità** coeso e coerente che va a comporre (insieme ad altri **service** e ai **repository** di dati) la logica di business dell'applicazione.

Per usare un **service** all'interno di un altro componente (potrebbe essere un **controller**, o un altro **service**, o come vedremo un **repository**) si usa la **dependency injection**

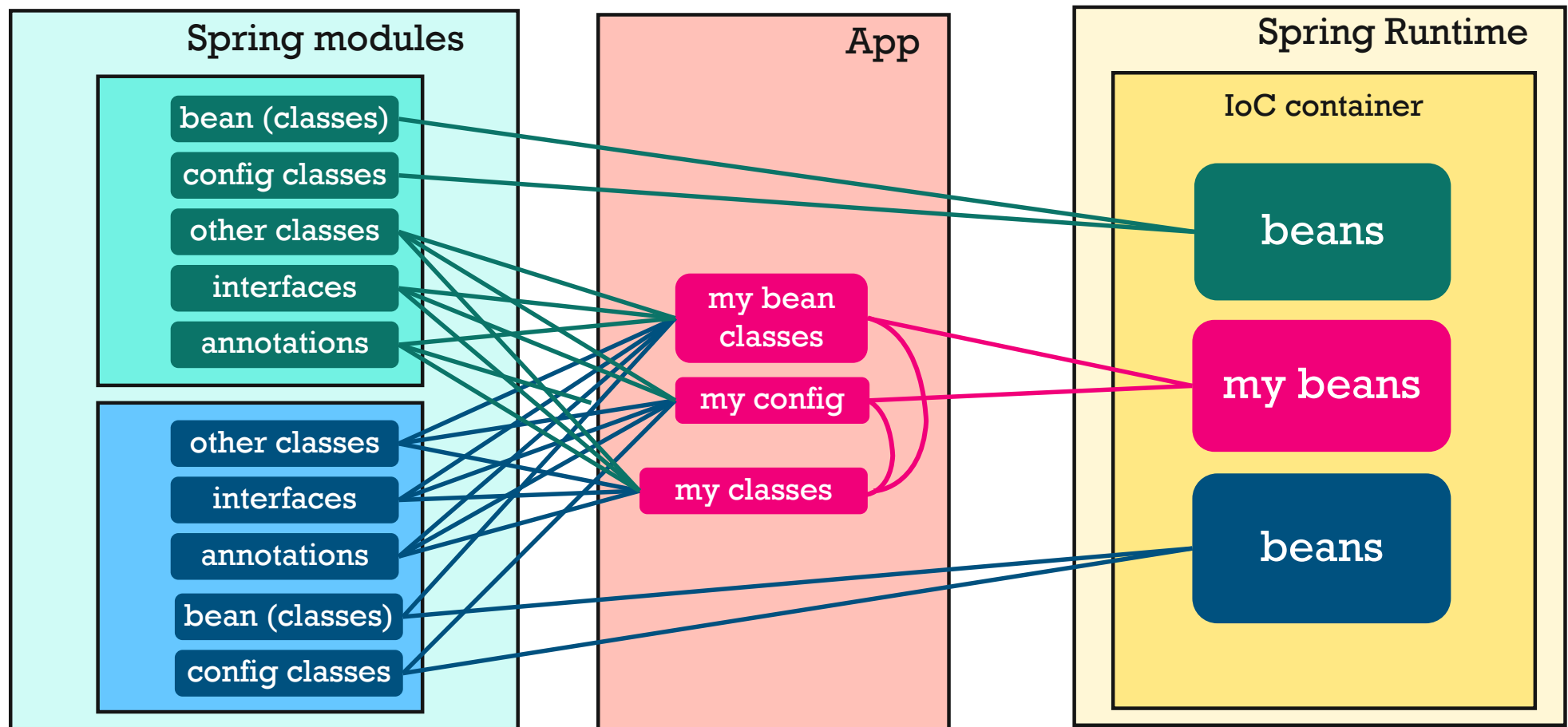
Spring prevede diverse forme di **dependency injection**, quella più comunemente utilizzata consiste semplicemente nel prevedere il **service** da iniettare come **parametro del costruttore** del componente che lo deve ricevere. Il codice del costruttore dovrà poi curare di memorizzarlo in un campo per poterlo utilizzare.

(*) **Emendamento!** I costruttori dei **service** (e dei **controller**, e dei **repository**) possono prendere parametri, purché si tratti di oggetti **iniettabili** quali, come abbiamo appena visto, i **service** stessi. A patto di non avere **dipendenze circolari**!

L'architettura di Spring

Uno sguardo ai meccanismi interni del framework

In che contesto viene eseguito il nostro codice? Come interagisce con il framework?



L'Architettura di Spring - I

Spring è un **ambiente di esecuzione** modulare al centro del quale si trova l'**Inversion of Control container (IOC container)**, che crea e gestisce oggetti chiamati **bean**. I **bean** sono responsabili delle funzionalità del **modulo**. Il programmatore agisce (a) programmando le proprie classi-*bean* (dette anche **componenti**), (b) fornendo estensioni di classi-*bean* esistenti (quindi, estensioni di componenti), (c) configurando oggetti-*bean* già previsti da Spring.

Cos'è un *bean*?

Un *bean* è un oggetto in esecuzione, che assolve a funzionalità richieste da un modulo Spring per portare a termine i propri compiti. Per esempio, pensando al modulo Spring Web, un `@RestController` è un *bean*, usato da Spring per rispondere alle richieste HTTP che pervengono su determinate route. Anche un `@Service` è un *bean*, che però non viene usato direttamente da Spring, bensì dai `@RestController` o da altri `@Service` definiti all'interno dell'applicazione. Nel modulo Spring Web sono dichiarati altri *bean* che gestiscono alcuni aspetti in modo predefinito, come ad esempio l'invio di una risposta HTTP «500 Internal Server Error» se l'esecuzione del nostro codice genera un errore.

Una prima particolarità dei *bean* è che sono tutti **singleton**: esiste sempre in esecuzione un solo *bean* di una data classe. Quindi spesso per indicare un *bean* si indica la sua classe di appartenenza: non è infatti possibile che nell'ambiente Spring convivano due oggetti di quella classe (se cercassimo di crearne due, cosa peraltro non semplice da far accadere perché Spring gestisce autonomamente la creazione dei bean, ciò risulterebbe in un'eccezione). L'altra caratteristica che contraddistingue i *bean* è che l'IOC container è in grado di **iniettare** un *bean* (passarlo come parametro) ad altri *bean* che ne fanno richiesta.

L'Architettura di Spring - II

IoC container

L'IoC container è il *core* di Spring. Esso è il modulo centrale. **La sua principale responsabilità è creare e gestire i *bean*.**

Tutti i *bean* dichiarati tali vengono creati dall'IoC container all'avvio dell'applicazione, rispettando eventuali dipendenze dichiarate (tramite il meccanismo della **dependency injection**) fra di essi. Infatti, se si creano dei circoli viziosi di dipendenze (A dipende da B che dipende da C che dipende da A) verrà lanciata un'eccezione.

Ci sono due modi di principali di dichiarare un *bean* affinché l'IoC container lo crei in fase di avvio dell'applicazione:

- **annotare una classe con `@Component`, o con una delle altre annotazioni derivate da essa** (`@RestController`, `@Service`, `@Repository`, `@Configuration` sono solo alcune delle annotazioni derivate da `@Component`). Questa sarà una classe-*bean* e l'IoC container ne creerà una singola istanza all'avvio. I parametri del costruttore della classe vengono utilizzati per stabilire le dipendenze da altri *bean*: essi verranno creati *prima* e successivamente passati al costruttore (sempre dall'IoC container) al momento della creazione. La dependency injection avviene qui a livello di costruttore. Attenzione: questo metodo di annotazione si applica anche a classi che **estendono** altre classi-*bean* già definiti nelle librerie di Spring.
- **annotare una classe con `@Configuration`, e uno o più dei suoi metodi con `@Bean`.** L'IoC container interpreta questo come «il metodo crea/restituisce un *bean*» e dunque lo esegue in fase di inizializzazione, inserendo l'oggetto fra i *bean* disponibili. In questo caso le eventuali dipendenze del *bean* costruito dal metodo vengono dichiarate fra i parametri del metodo stesso. La dependency injection dunque avviene qui a livello di metodo.

[...continua]

L'Architettura di Spring - III

Il ruolo dei moduli

Un **modulo** in Spring (es.: Spring Web, JPA, SpringSecurity) è un insieme di package (quindi: classi, interfacce, annotazioni) che **estende** il funzionamento base di Spring in una serie di modi:

- **fornisce bean infrastrutturali che offrono un comportamento di base.** Ad esempio Spring Web offre un bean `DispatcherServlet` che ascolta e riceve le richieste HTTP e le redirige ai `@RestController` e agli *handler* appropriati (o restituisce 404-Not found). Questo *bean* esiste indipendentemente dal fatto che noi definiamo o meno dei `@RestController`.
- **fornisce bean di configurazione che influenzano il comportamento dei bean infrastrutturali.** Ad esempio, fornisce l'insieme di *bean* responsabili per l'interpretazione dei *body* delle HTTP Request, e l'insieme di *bean* responsabili per la serializzazione dei *body* delle HTTP Response.
- **fornisce insiemi di annotazioni che permettono di estendere il comportamento di base con dei propri bean e di farli comunicare con i bean infrastrutturali e di configurazione.** Questo è ciò che Spring Web ad esempio fa fornendo le annotazioni `@RestController`, `@GetMapping`, `@PathVariable` ecc.

Programmare in Spring

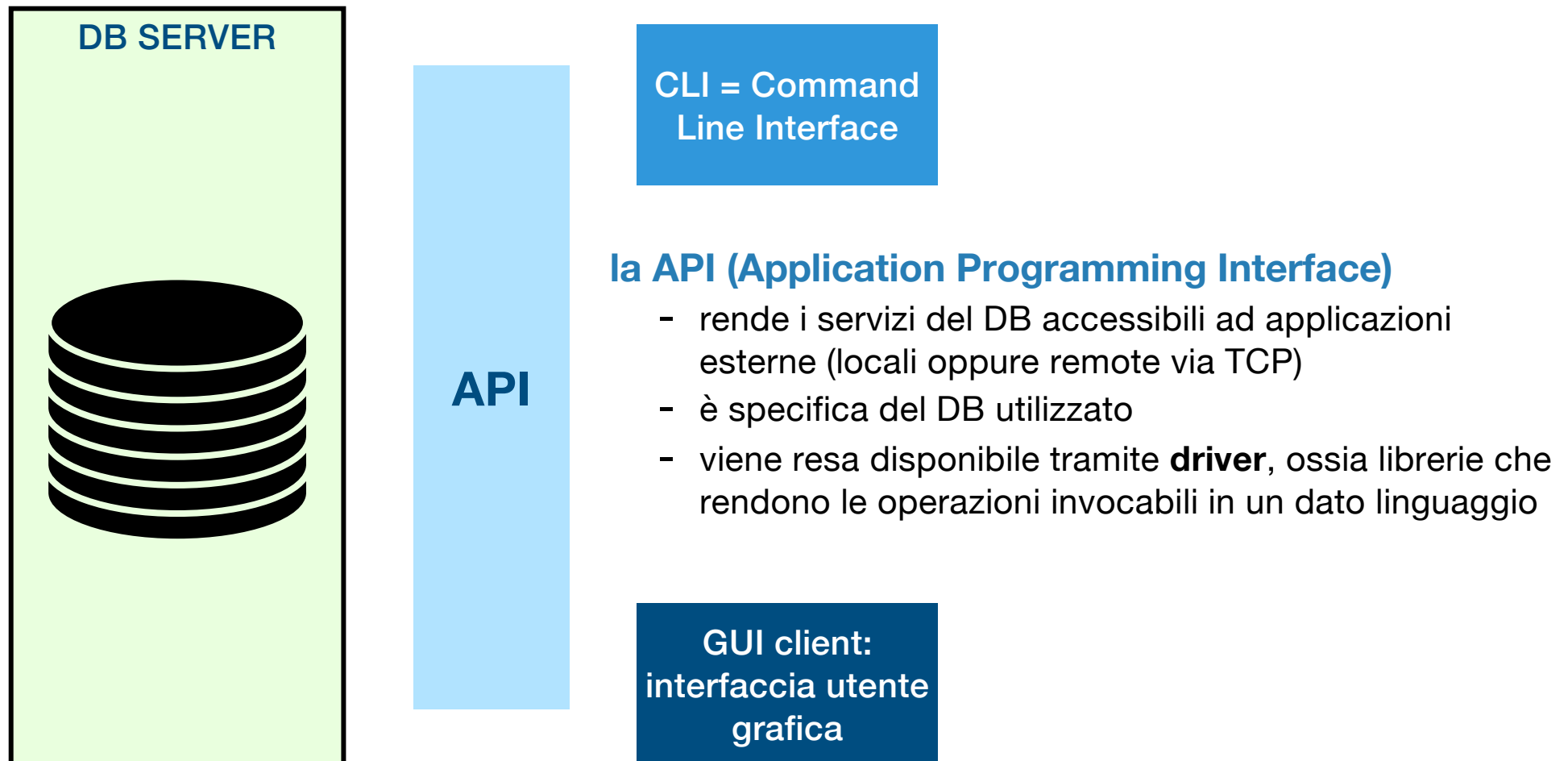
Programmare in Spring significa dunque essenzialmente due cose:

- **scrivere dei propri bean** che interagiscano con i *bean* esistenti, usando le annotazioni e dunque i meccanismi forniti dai moduli importati nel proprio progetto
- **modificare** (estendendoli, ridefinendoli o impostandoli attraverso l'invocazione di metodi) i *bean* di configurazione in modo da ottenere il comportamento desiderato.

Interfacciarsi a un DB relazionale

Come usiamo i suoi servizi?

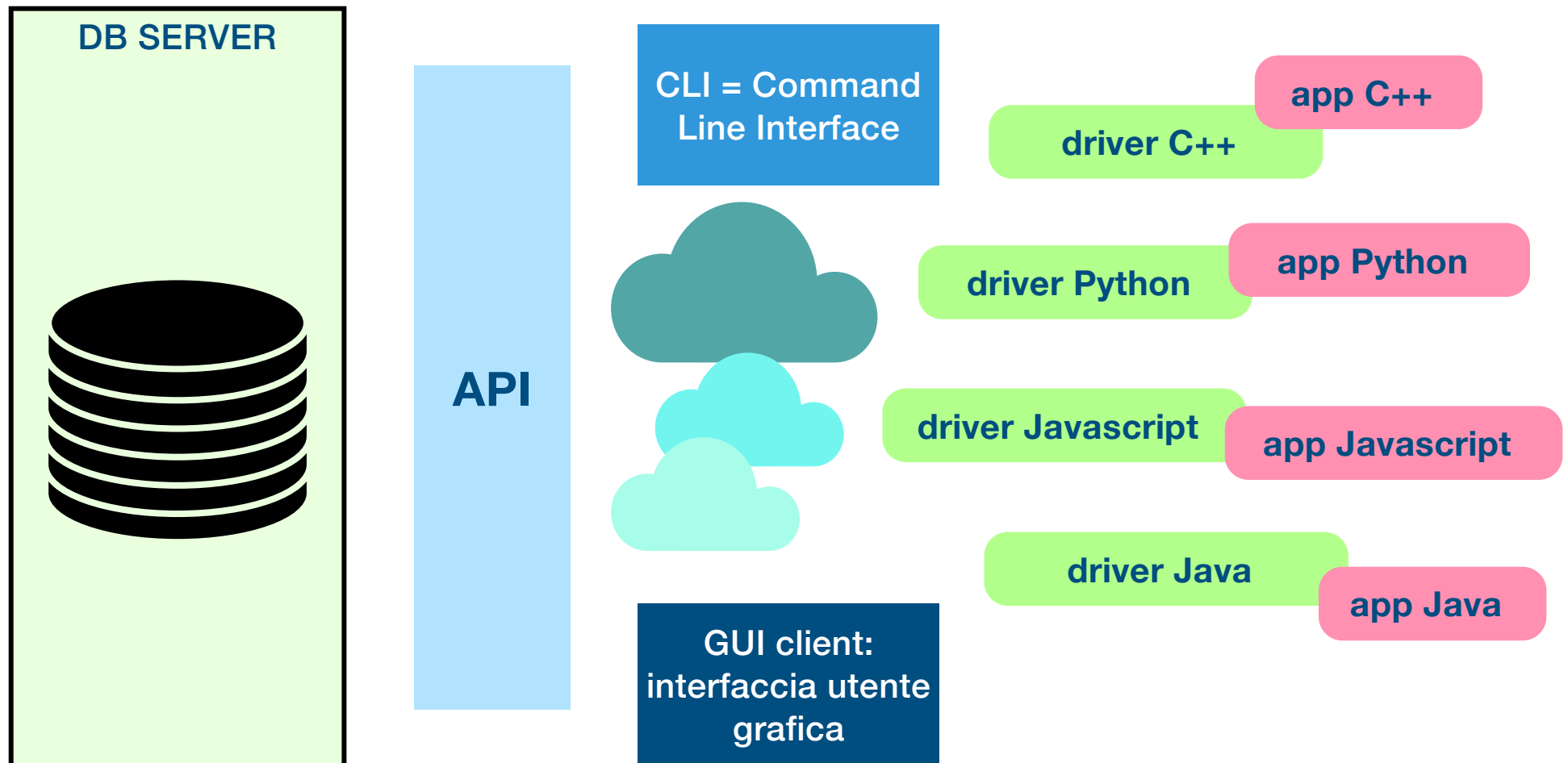
Ossia: come eseguiamo delle **query SQL** e ne leggiamo i risultati?



Interfacciarsi a un DB relazionale

Come usiamo i suoi servizi?

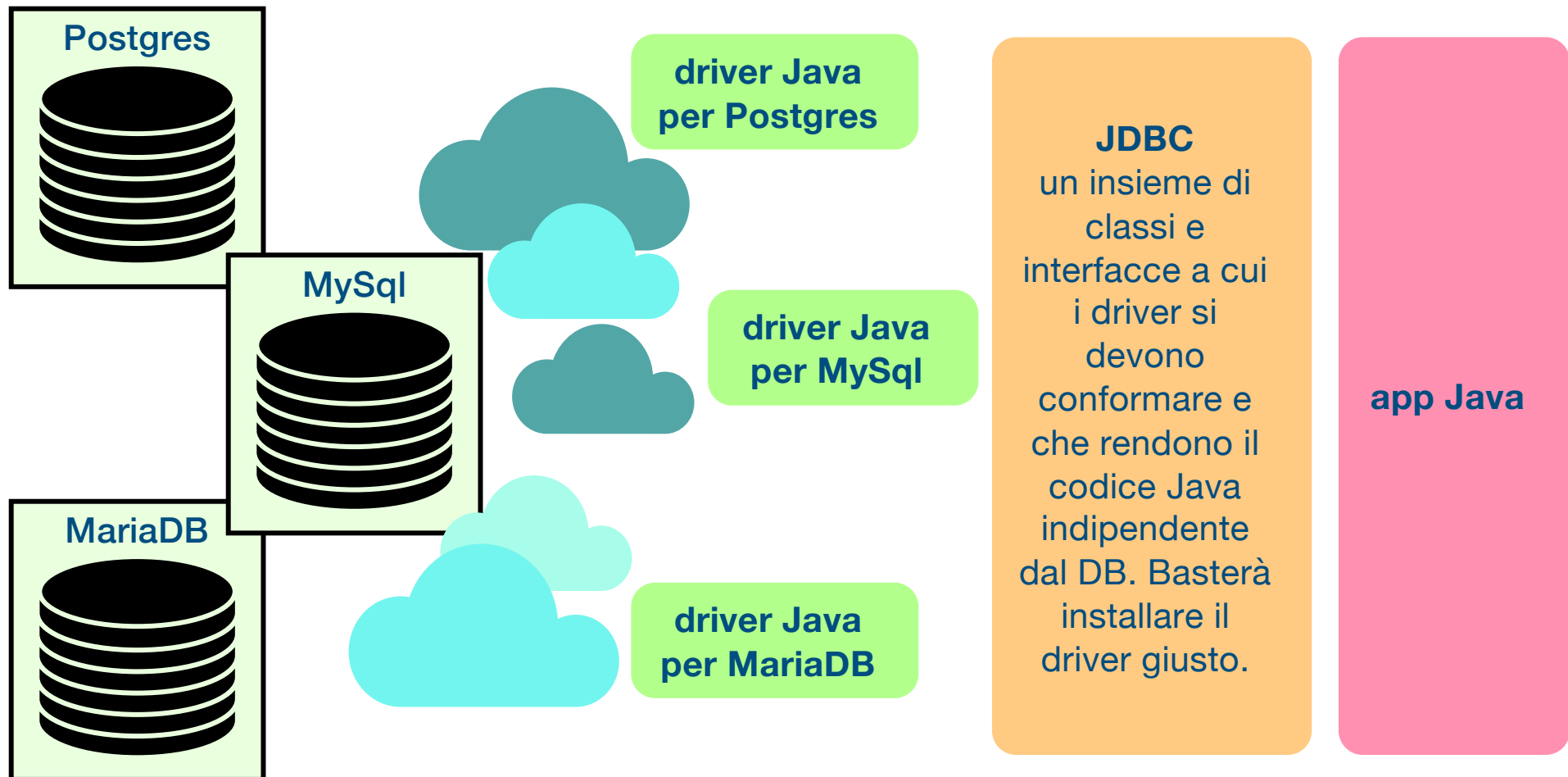
Ossia: come eseguiamo delle **query SQL** e ne leggiamo i risultati?



Interfacciarsi a un DB relazionale

Come usiamo i suoi servizi?

Ossia: come eseguiamo delle **query SQL** e ne leggiamo i risultati?



Interfacciarsi a un DB service

Come usiamo i suoi servizi?

Spring

Spring App

Jakarta Persistent API (JPA)

Hibernate ORM

JDBC

ORM

L'ORM (Object-Relational Mapping) è una tecnica utilizzata per mappare i costrutti principali dei DB relazionali in costrutti di programmazione orientata agli oggetti. Uno strumento di ORM fornisce una API che permette di accedere ai servizi di **persistenza dei dati** nel linguaggio di programmazione prescelto, utilizzando un modello orientato agli oggetti (anziché relazionale) e accedendo in lettura e scrittura ai dati del DB attraverso la creazione o eliminazione di oggetti e l'invocazione di metodi sugli stessi, anziché attraverso l'invocazione di query SQL sul DB.

JPA

JPA (Jakarta Persistence API) è parte di Jakarta EE ed è una specifica che descrive come gestire dati relazionali a partire da Java. JPA di per sé non implementa un servizio di ORM, è piuttosto una **specifica dell'interfaccia di programmazione** che un ORM per Java dovrebbe rispettare per essere usato nell'ambito di applicazioni JEE (il che include le applicazioni di Spring Boot). Ci possono essere diversi strumenti di ORM per Java; se tutti implementano la specifica JPA, chi sviluppa l'applicazione potrà utilizzare l'API di JPA senza essere vincolata ad uno specifico **fornitore (provider)**.

Hibernate

Hibernate è il **provider** di JPA preferito da Spring Boot. Hibernate esiste anche come ORM indipendente. Va precisato che noi non vedremo come usare Hibernate in sé, bensì come usare JPA per costruire e gestire un semplice database da collegare alla nostra applicazione server-side. Quindi in linea di principio si potrebbe anche usare un provider diverso. La cosa più semplice però in Spring Boot è richiedere tramite **Maven** il set di dipendenze **spring-boot-starter-data-jpa** che include proprio **Hibernate** come provider di JPA.

DB driver

Hibernate interagisce con il DB tramite **JDBC**; è dunque necessario includere fra le dipendenze anche i **Driver JDBC** del **DBMS** che scegliamo di utilizzare. Negli esempi che vedremo a lezione useremo **Postgres**.

JPA

Entità e relazioni

Una **entità** del dominio dell'applicazione in JPA corrisponde a una **classe** e viene tipicamente mappata su una **tabella** nel DB. La classe deve **necessariamente avere** un **costruttore di default** (*)

Le **istanze** della **classe** sono mappate su uno o più **record** (che possono coinvolgere quella **tabella** o anche altre, in caso di campi che esprimono relazioni con altre **entity**). I **campi** della classe esprimono proprietà dell'entità che possono essere rappresentate come **colonne** della tabella corrispondente, o come tabelle relazionali.

Quando programmiamo con JPA dobbiamo descrivere le **entity** (tramite classi opportunamente annotate) e le relative **relationship** (tramite altre classi o opportune annotazioni all'interno delle **entity**).

Non dobbiamo preoccuparci di tradurre ciò in «tabelle di database relazionale» perché della traduzione si occupa JPA.

Tuttavia è utile conoscere i meccanismi della rappresentazione sottostante per sfruttare al meglio le funzionalità di JPA

(*) Nulla impedisce che la classe abbia altri costruttori, che potremo usare noi per creare dal nostro codice istanze della classe. Quando però JPA dovrà creare automaticamente le istanze in caricamento dal DB, userà il costruttore di default per poi impostare i campi.

JPA

Descrivere una entità e le sue relazioni

- **@Entity** **annota la classe** per comunicare che si tratta di un'entità del DB che dovrà essere gestita da JPA
- **@Table(name = "table_name")** **annota la classe** indicando i dettagli della **tabella** che offrirà la persistenza di questa unità. Solitamente usata per specificare il **nome** della tabella.
- **@Id** **annota un campo** indicandolo come chiave primaria
- **@GeneratedValue(strategy = GenerationType.STRATEGY)** **annota ulteriormente il campo chiave primaria** indicando che la chiave è generata dal DBMS, e specificando la strategia che dovrà utilizzare per generarla. Per semplicità utilizzeremo la strategia **GenerationType.AUTO** che demanda al DBMS la scelta.
- **@Column** **annota un campo** indicando i dettagli della colonna ad esso corrispondente
 - **name = "col_name"** specifica il nome della colonna, se diverso da quello del campo
 - **nullable = true/false** specifica se la colonna supporta il valore **null**
 - **unique = true/false** specifica se i valori nella colonna devono essere unici
- Per una relazione Uno-Molti fra due entità A (l'uno) e B (i molti), è necessario **annotare un campo** in B con **@ManyToOne** e **annotare un campo** simmetrico in A con **@OneToMany**
 - il campo **@ManyToOne** in B dovrà essere di tipo A
 - il campo **@OneToMany** in A dovrà essere una Collection di oggetti B. L'annotazione **@OneToMany** dovrà specificare la proprietà **mappedBy = "owner"** dove owner è il nome del campo di B annotato con **@ManyToOne**
- Per una relazione Molti-Molti fra A e B è necessario **annotare campi simmetrici** in entrambe le **@Entity**, entrambi con **@ManyToMany**, entrambi Collection (meglio Set che List)
 - uno fra A e B dovrà essere l'**owner** e aggiungere anche un'annotazione **@JoinColumn** che specifica la join table da utilizzare
 - l'altro dovrà indicare tramite la proprietà **mappedBy = "owner"** il nome del campo dell'altra Entity che «possiede» la relazione

JPA

Repositories e operazioni CRUD

Per **operare** su una **tabella** del **DB**, dobbiamo definire un Repository. Un `@Repository` è una **interface** che **estende** `JpaRepository<E, Id>`

L'interfaccia `JpaRepository` definisce già i metodi di base per operazioni CRUD sul DB, dunque in molti casi potremo lasciare **vuota** la nostra estensione. `JpaRepository` è un tipo **generico**, parametrico rispetto alla **classe E** che descrive la **entity**, e al **tipo Id** della relativa **chiave primaria**.

- `E save(E entity)` salva una singola **entity** nella tabella corrispondente e restituisce l'entity risultante (che potrebbe essere diversa da quella di partenza). Se l'**id** di E è già presente nel DB questa operazione è una **update**; altrimenti è una **create**.
- `Iterable<E> saveAll(Iterable<E>)` salva un elenco di **entities** nella tabella e le restituisce.
- `Iterable<E> findAll()` restituisce tutte le **entities** nella tabella corrispondente
- `Optional<E> findById(Id id)` restituisce la **entity** con l'**id** specificato, se presente.
- `void deleteById(Id id)` elimina dalla tabella la **entity** con l'**id** specificato, se presente.
- `void delete(E entity)` elimina dalla tabella la **entity** indicata, se presente.

Per operare con altre **query** possiamo sfruttare le **query derivate** di JPA.

JPA

Query derivate

Se aggiungiamo al @Repository **dichiarazioni di metodi** che seguono una ben precisa **naming convention**, JPA è in grado di **derivare automaticamente** la **query SQL** necessaria ad implementare i metodi in questione.

```
findByAuthorAndTitleContainsOrderByTitleAsc(String author, String title)
deleteTop1ByAuthorBeforeIgnoreCase(String author)
countDistinctByTitleStartsWith(String title)
```

- **operazione**: cosa si vuole fare (sostanzialmente, una di queste tre...)
- **modificatore dell'operazione**: limita o condiziona i risultati
- **By**: separa il **soggetto** della query (**operazione** ed eventuale **modificatore**) dal **predicato**.
- **specificatore di predicato**: specifica una colonna e un'operazione logica (se omessa, è l'uguaglianza) per filtrare i dati sulla base del valore di quella colonna. Per ogni predicato dovrà esserci un parametro in ingresso che specifica il valore rispetto a cui verificare la condizione.
- **operatore logico**: permette di combinare più specificatori in un solo predicato composito (può essere **And** oppure **Or**)
- **modificatore del predicato**: **IgnoreCase** modifica l'applicazione dello **specificatore** che lo precede ignorando le maiuscole; **AllIgnoreCase** modifica l'applicazione dell'intero **predicato**; **OrderBy** (seguito dal nome di una o più colonne e dalla specifica **Asc** o **Desc**) fa sì che i risultati siano ordinati secondo il criterio dato prima di essere restituiti.

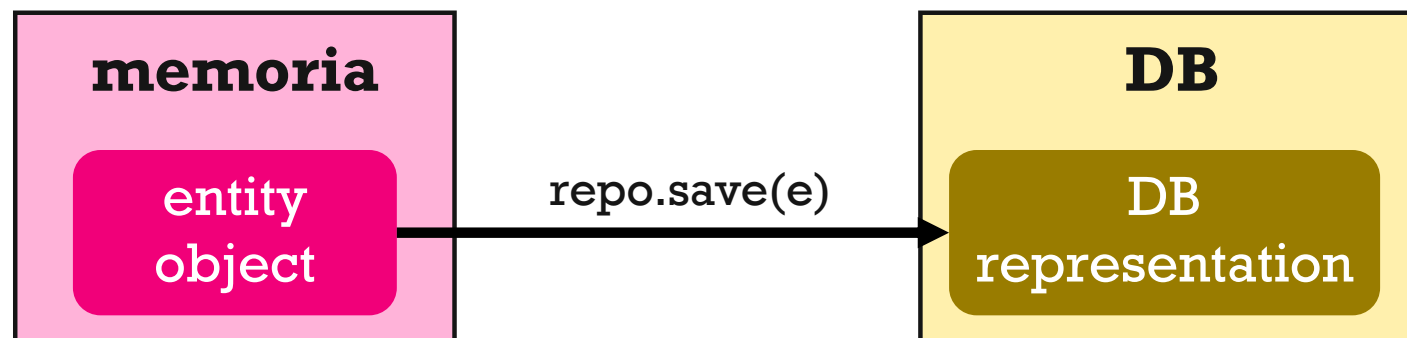
JPA

Ciclo di vita delle entità - 1

Quando un'entità viene creata in memoria, è **nuova** (new), JPA non la gestisce e il DB non ha idea che esista.



Quando un'entità viene salvata tramite **repo.save(e)** viene effettuata una **create**, e l'entità diventa **managed**, ossia gestita.
Lo resta finché non termina la **transazione** corrente.

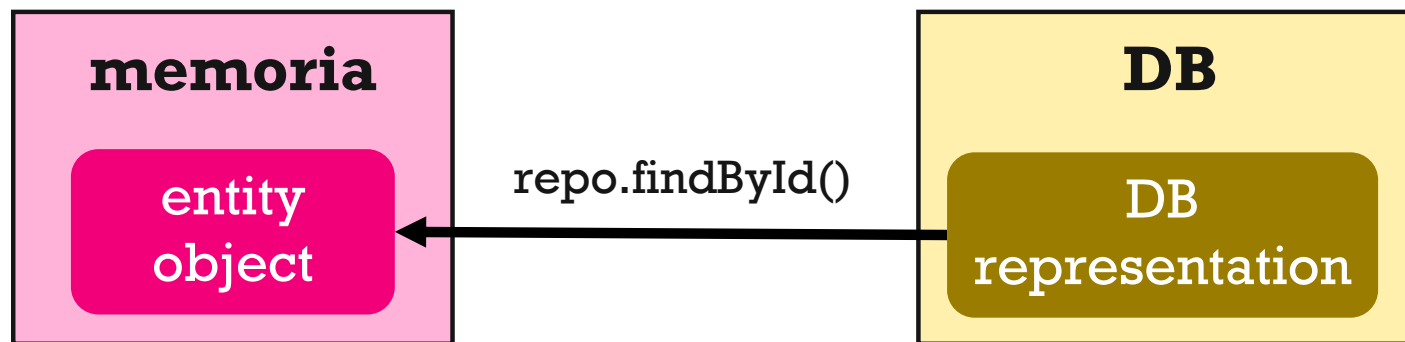


Quando un'entità è **managed** tutte le modifiche ai campi dell'oggetto si riflettono nel DB.

JPA

Ciclo di vita delle entità - 2

Anche quando estraiamo un'entità dal DB tramite uno dei metodi del **repository** (per esempio, **findById**) l'entità è **managed** fino al termine della **transazione**.



Al termine della **transazione** corrente, l'entità diventa **detached**, «staccata».

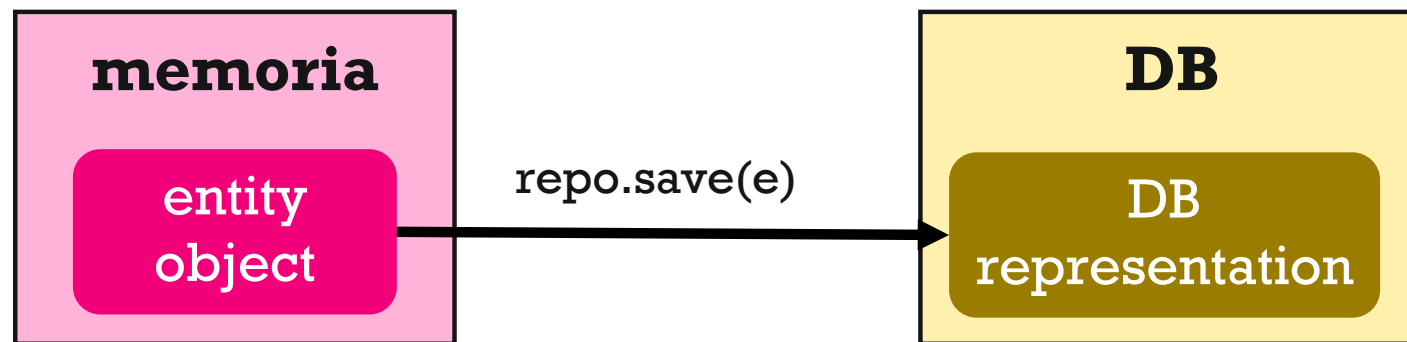


Le modifiche all'entità non si riflettono più nel DB.

JPA

Ciclo di vita delle entità - 3

Se voglio riagganciare un'entità **detached**, posso chiamare di nuovo **save**. Poiché l'entità ha un id presente nel DB, la **save** effettuerà **update** (anziché **create**)



save può fare una di due cose:

- **persist** se l'entità non ha Id; in questo caso aggiorna l'oggetto direttamente l'oggetto fornito in input
- **merge** se l'entità ha già un Id; in questo caso recupera dal DB una versione **managed** dell'entità, la aggiorna con i valori della versione **detached**, e restituisce la nuova entità recuperata

Quindi attenzione: in alcuni casi **save** restituisce lo stesso oggetto che riceve in input, in altri no!

JPA

Ciclo di vita delle entità - 4

Quando cancelliamo un'entità con **repo.delete** l'entità diventa **removed**: non appena la **transazione** termina, essa scomparirà dal DB



Abbiamo detto spesso «quando la **transazione** termina». Ma quando termina una transazione JPA?

- se non altrimenti specificato, la transazione è sempre limitata **ai singoli metodi del repository**
- quindi se chiamo **repo.save(e)** durante l'esecuzione di **save** l'entità è **managed**, ma appena **save** la restituisce, non lo è più!
- stessa cosa per **findById** e altri metodi di ricerca nel **repository**
- per avere una transazione più duratura, è necessario utilizzare l'annotazione **@Transactional**

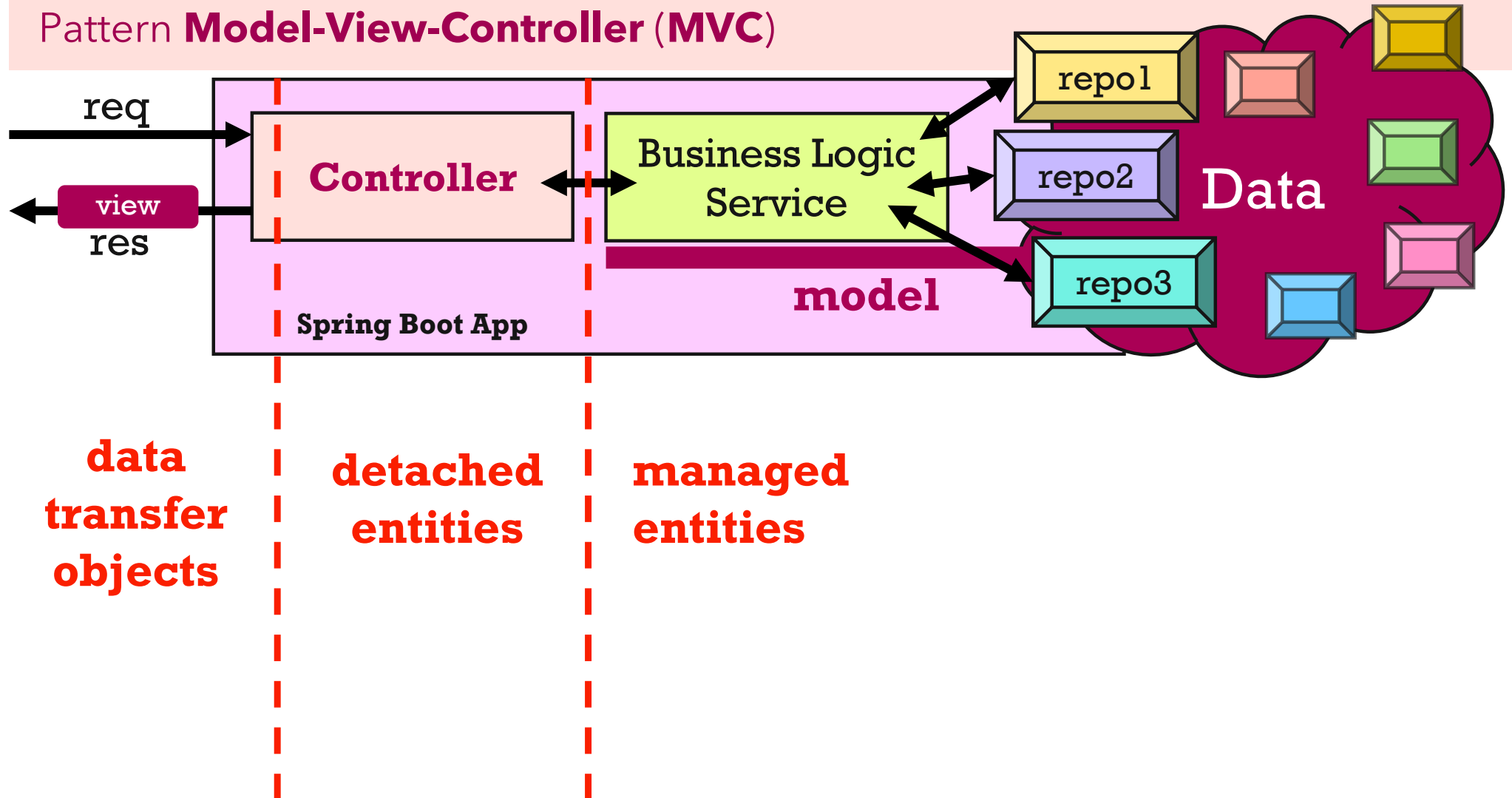
Un **metodo** @Transactional garantisce che le entità restino **managed** fino al termine del metodo stesso (e dunque le modifiche operate su di esse vengano o trasferite nel DB (**committed**) al termine del metodo, o annullate se il metodo fallisce (**rollback**))

- I **metodi dei un @Service** sono tipicamente **@Transactional**
- L'esecuzione di un metodo **@Transactional** è effettivamente incapsulata in una **transazione** solo se il metodo è invocato **da un altro componente** che **inietta** il servizio in questione tramite **Dependency Injection**
- Solo in questo modo il metodo infatti viene eseguito «sotto il controllo» di Spring, che può dunque farlo precedere e seguire dalle operazioni necessarie a garantire la transazionalità delle richieste al DB che lo compongono.

Spring-Boot

Entity e Data Transfer Object

Pattern **Model-View-Controller (MVC)**



Applicazione server-side di base con Spring Boot - I

Suddivisione delle responsabilità

In un'applicazione server-side di base realizzata con Spring Boot è importante dividere le responsabilità correttamente fra i diversi componenti che la costituiscono.

- I `@RestController` hanno tipicamente la responsabilità di **gestire le richieste HTTP** e scegliere, in base ai parametri (siano essi `@PathVariable` o `@RequestParam`), al `@RequestBody` se esistente, ed eventualmente ad altre caratteristiche della richiesta, quali funzionalità di **business logic** devono essere attivate. È anche responsabilità dei `@RestController` decidere, in base alla risposta di business logic ottenuta, che tipo di risposta inviare e quali dei dati ottenuti costituiranno il body della risposta. In sostanza, possiamo dire che è responsabilità dei `@RestController` garantire che l'API dell'applicazione server-side sia quella che è stata concordata con l'applicazione client-side.
- I `@Service` hanno la responsabilità di attuare i servizi di business logic, ossia le vere e proprie funzionalità dell'applicazione. I `@Service`, come detto precedentemente, forniscono i propri servizi ai `@RestController`, ma anche l'un l'altro: non è infrequente che un `@Service` utilizzi le funzionalità di un altro `@Service` per realizzare le proprie.
Frequentemente i `@Service` agiscono attuando qualche forma di I/O sullo strato di persistenza dei dati. A questo scopo, i `@Service` utilizzano le funzionalità dei `@Repository`: talvolta per realizzare una funzionalità complessa, un `@Service` avrà bisogno di cucire insieme più richieste diverse a più repository diversi.
- I `@Repository` hanno la responsabilità di garantire la corretta persistenza dei dati a loro affidati. Ogni `@Repository` gestisce una entità ben precisa. I `@Repository` non sanno nulla delle funzionalità di business logic (né tantomeno della server-side API); ciascuno conosce solo le caratteristiche della tabella a cui fa riferimento.

Applicazione server-side di base con Spring Boot - II

Visione delle entità e dei dati

La rappresentazione delle entità che viene espressa nella persistenza (e dunque attraverso le `@Entity` e i `@Repository` di JPA) è utilizzata dalla **business logic di back-end**, ma non è necessariamente la stessa che viene utilizzata nella **comunicazione con il front-end**. Anzi, quasi mai è la stessa e *non si dovrebbe ipotizzare* che sia la stessa. I motivi sono vari:

- A volte c'è proprio una «visione» del modello diversa fra back-end e front-end; le entità che il front-end mostra e manipola sono *collegate* a quelle rappresentate e persistite dal back-end ma non sono le stesse.
- A volte il back-end non vuole esporre il modello dati così com'è, ma magari nasconderne delle parti (si pensi alla password dell'oggetto «User»!).
- Dal momento che le `@Entity` in JPA sono spesso collegate fra di loro tramite le relazioni `@OneToMany`, `@ManyToMany`, ecc, gli oggetti corrispondenti sono spesso molto ricchi, contengono collegamenti ad altri oggetti, che a loro volta si collegano ad altri oggetti, ecc. Spesso nel restituire una risposta al front-end si vuole limitare la profondità e la complessità di questi oggetti, e mandare una versione più «shallow», riservandosi di mandare ulteriori dati solo se il front-end ne fa richiesta.
- A volte, viceversa, si vogliono aggregare più `@Entity` lato back-end in un unico oggetto che combina gli aspetti che interessano al front-end al tempo stesso mettendo da parte quelli non pertinenti

Le classi che definiamo per rappresentare la «visione» sulle entità da trasferire al front-end sono dette Data Transfer Object. È in generale opportuno definire i propri DTO. La responsabilità della conversione fra Entity e DTO è del Controller, ma la prassi più comune è definire una classe «DTOMapper» con metodi di conversione a cui il Controller può appoggiarsi.